The Journal of Open Source Software

Magba: Analytical Magnetostatic Computation for Rust

Sira Pornsiriprasert [1¶]

1 Faculty of Medicine Ramathibodi Hospital, Mahidol University, Thailand ¶ Corresponding author

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor:

Submitted: 01 July 2026
Published: unpublished

Summary

Magba is an analytical magnetostatic computation library designed for high-performance and seamless integration into the Rust scientific computing ecosystem. Leveraging zero-cost abstractions, it provides highly parallelized routines for evaluating magnetic fields generated by permanent magnets and current loops. Magba allows researchers to construct and manipulate complex magnetic systems with low runtime overhead.

Statement of Need

Accurate and fast computation of static magnetic fields is essential in numerous scientific and engineering domains, including sensor design (Jones et al., 2020), magnetic resonance imaging (MRI) (Koch et al., 2009), and robotics (Song et al., 2018). While the finite element method (FEM) provides versatile solutions for complex geometries, it is computationally intensive and often unsuitable for real-time applications. Analytical solutions for canonical magnet geometries provide a substantially faster alternative, yet relatively few software packages implement them. The most notable instance is Magpylib (Ortner & Coliado Bandeira, 2020), a Python library for analytical field computation. However, its reliance on the Python interpreter can bottleneck performance-critical workloads, such as optimization algorithms in magnetic tracking systems.

Magba is a Rust library for high-performance magnetostatic field computation based on analytical solutions. It provides parallelized magnetic field calculation routines with low memory overhead while exposing a high-level, object-oriented API for constructing and manipulating magnetic systems. Additionally, the library provides physical sensor implementations, such as linear Hall effect sensors, that can be integrated into magnetic systems to measure magnetic fields and simulate readings.

Software Implementation

Field functions are implemented based on analytical solutions from the literature (e.g., Caciagli et al. (2018); Derby & Olbert (2010); Ortner et al. (2023)), as well as fundamental expressions, such as the Biot-Savart law and the magnetic dipole equations (Furlani, 2001). The implementations utilize a custom generic float trait that provides magnetic constants and integrates with the mathematical backends. All physical quantities are assumed to be in SI units.

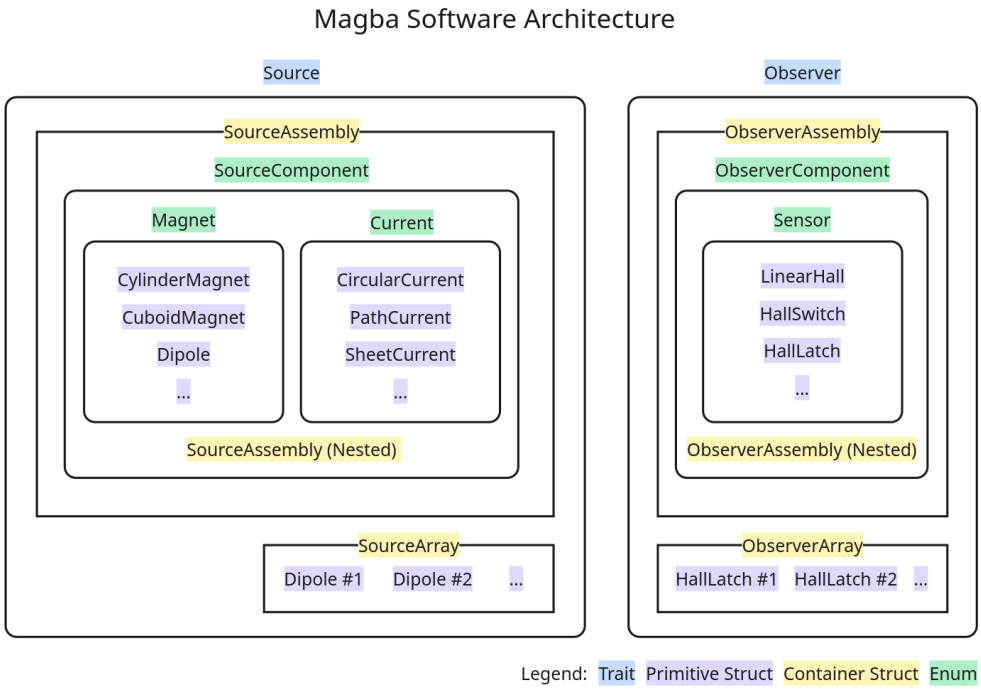

Figure 1: Core software architecture of the Magba library.

Magba represents physical entities using algebraic data types, expresses behavior through trait-based closed polymorphism, and organizes entities using a hierarchical composite model (Figure 1). This design avoids dynamic dispatch for built-in entity types while preserving extensibility through hierarchical composition. Concrete types implement core traits `Source` and `Observer`, corresponding to objects that generate and measure magnetic fields, respectively. All objects support spatial manipulation through integration with nalgebra (dimforge, 2026), a widely adopted linear algebra library in Rust.

The composition system provides two types of containers: arrays and assemblies. The arrays (`SourceArray`, `ObserverArray`) are fixed-sized, stack-allocated collections of homogeneous entities, while the assemblies (`SourceAssembly`, `ObserverAssembly`) are dynamically sized, heap-allocated collections of heterogeneous types, that support nesting and custom types by wrapping them in the `SourceComponent` and `ObserverComponent` enums. Each child in the collection is stored as a node that, in addition to holding the concrete entity, keeps track of the pose offsets relative to its parent, maintaining accuracy after repeated transformation.

Results

Magba was extensively evaluated against Magpylib's (Ortner & Coliado Bandeira, 2020) fully vectorized NumPy implementations. Test magnets were assigned a $[1, 2, 3]$ T polarization, and fields were computed on a 1,000-point grid spanning ± 0.5 m. Accuracy is measured using the relative Euclidean distance error.

Benchmarks were conducted on an AMD Ryzen 5 4600H 4.0 GHz, 16 GB of RAM, running x86_64-unknown-linux-gnu with rustc 1.90.0 and magba v0.6.1, utilizing Criterion (Aparicio et al., 2025) and asv (Droettboom et al., 2026) for performance profiling in Rust and Python, respectively. The reported compute times are normalized per observer point.

Table 1: Summary of Field Function Accuracy and Performance at 64-bit Precision.

Function	Median Error	Max Error	Magba	Magpylib	Speedup
Magnets					
Cylinder	2.947×10^{-13}	2.501×10^{-10}	63.3 ns	2.49 µs	39x
Cuboid	0.000	2.103×10^{-13}	136.3 ns	1.33 µs	10x
Dipole	0.000	0.000	31.6 ns	422 ns	13x
Sphere	0.000	8.078×10^{-16}	26.9 ns	518 ns	19x
Tetrahedron	0.000	1.020×10^{-10}	111.5 ns	4.07 µs	37x
Triangle	0.000	2.405×10^{-12}	55.6 ns	881 ns	16x
Mesh	0.000	1.020×10^{-10}	111.1 ns	7.53 µs	68x
Currents					
Circular	0.000	0.000	46.9 ns	739 ns	16x
Path	0.000	0.000	54.2 ns	1.87 µs	35x
Sheet	0.000	0.000	208.1 ns	19.4 µs	93x
Triangle	0.000	0.000	78.6 ns	10.6 µs	135x

The results closely align with Magpylib, with the median below one machine epsilon for most implementations. The slight variance observed in the cylinder magnet calculations is attributable to the different mathematical backends used to evaluate elliptic integrals, namely Ellip (Pornsiriprasert, 2026) versus SciPy (Virtanen et al., 2020). Computationally, Magba delivers a speedup of over an order of magnitude across all geometries. These gains are especially pronounced for mathematically intensive geometries, such as mesh magnets (68x) and triangle sheet currents (135x).

Usage Example

The following example demonstrates a magnetic field calculation generated by an assembly of cylindrical and cuboid magnets at a specific observer point.

```rust
use magba::{prelude::*, sources};

// Create magnets and group into a source assembly
let cylinder = CylinderMagnet::default()
    .with_diameter(0.05)
    .with_height(0.1);
let cuboid = CuboidMagnet::<f64>::default();
let mut assembly = sources!(cylinder, cuboid);

// Apply transformation on the source assembly
assembly.translate([0.0, 0.0, 0.1]);

// Compute the magnetic field at an observation point
let b_field = assembly.compute_B(nalgebra::Point3::new(0.0, 0.0, 0.3));

println!("B = [{}, {}, {}]", b_field[0], b_field[1], b_field[2]);
// B = [0, 0, 0.6316701187086277]
```

AI Usage Disclosure

The author conceptualized all software design decisions and testing frameworks. Gemini 3.1 Pro was utilized to assist with code generation, documentation drafting, and manuscript refinement. All AI-generated outputs were reviewed, edited, and validated by the author.

The Journal of Open Source Software

Acknowledgment

I thank Michael Ortner and the Magpylib contributors for the technical references and their unwavering commitment to the scientific community.

References

Aparicio, J., Himmelstrup, D., & Karchebnyi, B. (2025). *Criterion.rs* (Version 0.5.1). https://github.com/criterion-rs/criterion.rs

Caciagli, A., Baars, R. J., Philipse, A. P., & Kuipers, B. W. M. (2018). Exact expression for the magnetic field of a finite cylinder with arbitrary uniform magnetization. *Journal of Magnetism and Magnetic Materials*, *456*, 423–432. https://doi.org/10.1016/j.jmmm.2018.02.003

Derby, N., & Olbert, S. (2010). Cylindrical magnets and ideal solenoids. *American Journal of Physics*, *78*(3), 229–235. https://doi.org/10.1119/1.3256157

dimforge. (2026). *Nalgebra* (Version 0.34.2). https://nalgebra.rs/

Droettboom, M., Virtanen, P., & asv Developers. (2026). *Asv: Airspeed velocity* (Version 0.6.6). https://github.com/airspeed-velocity/asv

Furlani, E. P. (2001). *Permanent magnet and electromechanical devices: Materials, analysis, and applications*. Academic. ISBN: 978-0-12-269951-1

Jones, D., Wang, L., Ghanbari, A., Vardakastani, V., Kedgley, A. E., Gardiner, M. D., Vincent, T. L., Culmer, P. R., & Alazmani, A. (2020). Design and Evaluation of Magnetic Hall Effect Tactile Sensors for Use in Sensorized Splints. *Sensors*, *20*(4), 1123. https://doi.org/10.3390/s20041123

Koch, K. M., Rothman, D. L., & de Graaf, R. A. (2009). Optimization of static magnetic field homogeneity in the human and animal brain in vivo. *Progress in Nuclear Magnetic Resonance Spectroscopy*, *54*(2), 69–96. https://doi.org/10.1016/j.pnmrs.2008.04.001

Ortner, M., & Coliado Bandeira, L. G. (2020). Magpylib: A free Python package for magnetic field computation. *SoftwareX*, *11*, 100466. https://doi.org/10.1016/j.softx.2020.100466

Ortner, M., Leitner, P., & Slanovc, F. (2023). Numerically stable and computationally efficient expression for the magnetic field of a current loop. *Magnetism*, *3*(1), 11–31. https://doi.org/10.3390/magnetism3010002

Pornsiriprasert, S. (2026). Ellip: An elliptic integral library for Rust. *Journal of Open Source Software*, *11*(118), 9386. https://doi.org/10.21105/joss.09386

Song, S., Zhang, C., Liu, L., & Meng, M. Q.-H. (2018). Preliminary study on magnetic tracking-based planar shape sensing and navigation for flexible surgical robots in transoral surgery: Methods and phantom experiments. *International Journal of Computer Assisted Radiology and Surgery*, *13*(2), 241–251. https://doi.org/10.1007/s11548-017-1672-8

Virtanen, P., Gommers, R., Oliphant, T. E., Haberland, M., Reddy, T., Cournapeau, D., Burovski, E., Peterson, P., Weckesser, W., Bright, J., van der Walt, S. J., Brett, M., Wilson, J., Millman, K. J., Mayorov, N., Nelson, A. R. J., Jones, E., Kern, R., Larson, E., … van Mulbregt, P. (2020). SciPy 1.0: Fundamental algorithms for scientific computing in Python. *Nature Methods*, *17*(3), 261–272. https://doi.org/10.1038/s41592-019-0686-2